Aula 20261029 - Flask e Templates

Na Unidade 2, construímos uma API REST que guardava dispositivos em uma lista e respondia em JSON. Nesta aula, vamos dar um passo além: construir um **sistema completo de cadastro de computadores** para um sistema de monitoramento, com **duas formas de acesso**:

- **Páginas web**, montadas com os *templates* do Flask, para uma pessoa usar pelo navegador;
- uma **API REST**, que devolve JSON, para outros programas usarem (como o nosso cliente em modo texto).

As regras de acesso serão as seguintes:

Operação	Quem pode fazer	Como se identifica
Listar, buscar e consultar	Qualquer pessoa	Não precisa se identificar
Cadastrar, editar e apagar (páginas)	Somente quem fez login	Usuário e senha
Cadastrar, editar e apagar (API)	Somente quem tem o token	Cabeçalho `Authorization: Bearer <token>`

Objetivos da aula

- organizar um projeto Flask com a pasta `templates`;
- criar páginas HTML dinâmicas com o Jinja2 (`{{ }}`, `{% if %}`, `{% for %}` e herança de templates);
- receber dados de formulários HTML com `request.form`;
- implementar as operações CRUD tanto nas páginas quanto na API;
- proteger as operações de alteração com login (páginas) e token (API);
- consumir a API com um cliente em modo texto.

> **O que é CRUD?**
>
> É a sigla para as quatro operações básicas sobre dados: **C**reate (cadastrar), **R**ead (ler/consultar), **U**pdate (atualizar) e **D**elete (apagar). Quase todo sistema de cadastro é, no fundo, um CRUD.

1. Preparando o projeto

O Flask espera encontrar os templates em uma pasta chamada exatamente **`templates`**, ao lado do programa principal. Crie esta estrutura de pastas e arquivos:

```
monitor/
├── app.py              ← o servidor (páginas + API)
├── cliente.py          ← o cliente em modo texto
└── templates/
    ├── base.html       ← o "molde" comum a todas as páginas
    ├── lista.html      ← página inicial: tabela e busca
    ├── formulario.html ← cadastro e edição
    └── login.html      ← tela de login
```

Abra um terminal na pasta `monitor`, crie o ambiente virtual e instale as bibliotecas:

```
python -m venv .venv
source .venv/bin/activate        # Linux/macOS
# .venv\Scripts\Activate.ps1    # Windows (PowerShell)
python -m pip install flask requests
```

2. Planejando antes de programar

O recurso: computador

Cada computador será um dicionário Python com quatro campos:

Campo	Exemplo	Regra
`id`	`1`	Número criado pelo servidor; o usuário não escolhe

`ip`	`192.168.0.101`	Precisa ser um endereço IPv4 ou IPv6 válido
`mac`	`AA:BB:CC:00:00:02`	6 pares de dígitos hexadecimais separados por `:` ou `-`

As rotas

Vamos separar as rotas em dois grupos. As páginas respondem com HTML; as rotas que começam com `/api` respondem com JSON.

Método	Rota	O que faz	Protegida?
GET	`/`	Página com a lista e a busca	Não
GET / POST	`/login`	Mostra e processa o formulário de login	Não
GET	`/logout`	Sai do sistema	Não
GET / POST	`/computadores/novo`	Formulário de cadastro	**Login**
GET / POST	`/computadores/<id>/editar`	Formulário de edição	**Login**
POST	`/computadores/<id>/apagar`	Apaga o computador	**Login**
GET	`/api/computadores`	Lista em JSON (aceita `?busca=`)	Não
GET	`/api/computadores/<id>`	Um computador em JSON	Não
POST	`/api/computadores`	Cadastra	**Token**
PUT	`/api/computadores/<id>`	Atualiza	**Token**
DELETE	`/api/computadores/<id>`	Apaga	**Token**

3. Construindo o servidor (app.py)

Vamos escrever o `app.py` em partes. **Cada trecho deve ser acrescentado ao final do arquivo, na ordem em que aparece.** Ao final da seção, o arquivo estará completo.

3.1 Início: importações, configurações e os dados

app.py (parte 1 de 9)

```
# app.py – Sistema de cadastro de computadores para monitoramento
# Uma única aplicação Flask com duas "portas de entrada":
#   - páginas HTML (templates) para usar no navegador;
#   - uma API REST (JSON) para outros programas.
import re
from ipaddress import ip_address

from flask import Flask, flash, jsonify, redirect, render_template, request, session, url_for

app = Flask(__name__)
app.json.ensure_ascii = False      # mantém os acentos nas respostas JSON
app.secret_key = "troque-esta-chave"   # necessária para usar session e flash

# ---------------------------------------------------------------
# Credenciais (apenas para fins didáticos!)
# ---------------------------------------------------------------
USUARIO = "admin"
SENHA = "ifrn"
TOKEN_API = "token-da-turma"

# ---------------------------------------------------------------
# "Banco de dados": uma lista de dicionários na memória
# ---------------------------------------------------------------
computadores = [
    {"id": 1, "nome": "Servidor-Web", "ip": "192.168.0.10", "mac": "AA:BB:CC:00:00:01"},
    {"id": 2, "nome": "Lab-PC-01", "ip": "192.168.0.101", "mac": "AA:BB:CC:00:00:02"},
    {"id": 3, "nome": "Impressora-Sala", "ip": "192.168.0.50", "mac": "AA:BB:CC:00:00:03"},
]
proximo_id = 4
```

- `USUARIO`, `SENHA` e `TOKEN_API` são as credenciais. Deixá-las escritas no código é aceitável **só em um exemplo didático** (veja a seção 8).
- A lista `computadores` faz o papel de banco de dados. Os dados voltam ao estado inicial sempre que o servidor é reiniciado. Em outra aula, trocaremos a lista por um banco SQLite.
- `proximo_id` guarda o número que será dado ao próximo computador cadastrado.

3.2 Funções auxiliares

Estas funções serão usadas **tanto pelas páginas quanto pela API**. Assim, as regras (como validar um IP) ficam escritas em um só lugar.

app.py (parte 2 de 9)

```
# ----------------------------------------------------------------
# Funções auxiliares (usadas pelas páginas E pela API)
# ----------------------------------------------------------------
def buscar_por_id(id):
    for computador in computadores:
        if computador["id"] == id:
            return computador
    return None


def filtrar(texto):
    """Devolve os computadores cujo nome, IP ou MAC contém o texto."""
    if not texto:
        return computadores
    texto = texto.lower()
    resultado = []
    for c in computadores:
        if texto in c["nome"].lower() or texto in c["ip"] or texto in c["mac"].lower():
            resultado.append(c)
    return resultado


def validar(dados):
    """Confere os dados recebidos. Devolve uma mensagem de erro ou None."""
    if not isinstance(dados, dict):
        return "Os dados devem ser enviados em JSON."

    nome = str(dados.get("nome", "")).strip()
    ip = str(dados.get("ip", "")).strip()
    mac = str(dados.get("mac", "")).strip()

    if nome == "":
        return "O nome é obrigatório."
    try:
        ip_address(ip)
    except ValueError:
        return "Endereço IP inválido."
    if not re.fullmatch(r"([0-9A-Fa-f]{2}[:-]){5}[0-9A-Fa-f]{2}", mac):
        return "Endereço MAC inválido. Use o formato AA:BB:CC:DD:EE:FF."
    return None


def limpar(dados):
    """Padroniza os dados: tira espaços e escreve o MAC em maiúsculas com ':'."""
    return {
        "nome": dados["nome"].strip(),
        "ip": dados["ip"].strip(),
        "mac": dados["mac"].strip().upper().replace("-", ":"),
    }


def usuario_logado():
    return session.get("logado") is True


def token_valido():
    return request.headers.get("Authorization") == f"Bearer {TOKEN_API}"
```

`filtrar(texto)`	Devolve os computadores cujo nome, IP ou MAC *contém* o texto buscado, sem diferenciar maiúsculas de minúsculas. Se o texto estiver vazio, devolve todos.
`validar(dados)`	Confere os três campos. Devolve a **mensagem de erro** do primeiro problema encontrado, ou `None` se estiver tudo certo.
`limpar(dados)`	Padroniza os dados antes de salvar: tira espaços e escreve o MAC sempre como `AA:BB:CC:DD:EE:FF`.
`usuario_logado()`	Diz se o usuário do navegador fez login.
`token_valido()`	Diz se a requisição da API trouxe o token correto no cabeçalho `Authorization`.

> **Entendendo a validação do MAC**
>
> A expressão regular `([0-9A-Fa-f]{2}[:-]){5}[0-9A-Fa-f]{2}` pode ser lida assim: *dois dígitos hexadecimais seguidos de `:` ou `-`, repetido 5 vezes, e depois mais dois dígitos hexadecimais*. A função `re.fullmatch` só aceita se o texto **inteiro** seguir esse padrão. Já o IP é conferido pela função `ip_address`, que gera um `ValueError` quando o endereço é inválido.

4. Templates: páginas HTML dinâmicas

Um **template** é um arquivo HTML com "lacunas" que o Flask preenche antes de enviar a página ao navegador. Quem faz isso é o **Jinja2**, instalado junto com o Flask. Na rota, chamamos:

```
return render_template("lista.html", computadores=lista, busca=busca)
```

O Flask procura `lista.html` na pasta `templates`, preenche as lacunas com as variáveis informadas (`computadores` e `busca`) e devolve o HTML pronto.

A sintaxe do Jinja2 que vamos usar

Sintaxe	Para que serve	Exemplo
`{{ ... }}`	Escreve um valor na página	`{{ c.nome }}`
`{% if %} ... {% endif %}`	Mostra um trecho só se a condição for verdadeira	`{% if session.logado %}`
`{% for %} ... {% endfor %}`	Repete um trecho para cada item de uma lista	`{% for c in computadores %}`
`{% extends "base.html" %}`	Diz que a página usa outro template como molde	Primeira linha das páginas
`{% block nome %} ... {% endblock %}`	Define uma área que cada página preenche do seu jeito	`{% block conteudo %}`
`{{ lista|length }}`	Aplica um *filtro* ao valor (aqui, o tamanho da lista)	`{{ computadores|length }}`
`url_for('funcao')`	Gera o endereço de uma rota a partir do nome da função	`{{ url_for('login') }}`

> **Segurança automática**
>
> O `{{ }}` **escapa** o HTML: se alguém cadastrar um computador com o nome `<script>alert(1)</script>`, a página mostra esse texto em vez de executar o código. Isso protege contra um ataque chamado *XSS*.

4.1 O molde: base.html

Todas as páginas têm o mesmo topo (menu) e a mesma área de avisos. Em vez de repetir isso em cada arquivo, escrevemos uma vez no `base.html` e as outras páginas **herdam** dele.

templates/base.html

```
<head>
  <meta charset="utf-8">
  <title>Monitoramento - {% block titulo %}{% endblock %}</title>
  <style>
    body  { font-family: Arial, sans-serif; max-width: 900px; margin: 20px auto; padding: 0 10px; }
    nav   { background: #1f5f8b; padding: 10px; border-radius: 4px; }
    nav a { color: white; margin-right: 15px; text-decoration: none; }
    table { border-collapse: collapse; width: 100%; margin-top: 15px; }
    th, td { border: 1px solid #ccc; padding: 6px 8px; text-align: left; }
    th    { background: #eef3f7; }
    .aviso { background: #fff3cd; border: 1px solid #e0c060; padding: 8px; margin-top: 10px; }
    label { display: block; margin-top: 10px; }
    label input { display: block; }
    input { padding: 5px; width: 300px; }
    button { padding: 5px 12px; margin-top: 10px; }
  </style>
</head>
<body>
  <nav>
    <a href="{{ url_for('pagina_inicial') }}">Computadores</a>
    {% if session.logado %}
      <a href="{{ url_for('novo_computador') }}">+ Novo</a>
      <a href="{{ url_for('logout') }}">Sair</a>
    {% else %}
      <a href="{{ url_for('login') }}">Entrar</a>
    {% endif %}
  </nav>

  {% for mensagem in get_flashed_messages() %}
    <div class="aviso">{{ mensagem }}</div>
  {% endfor %}

  {% block conteudo %}{% endblock %}
</body>
</html>
```

- Os blocos `titulo` e `conteudo` são as "lacunas" que cada página vai preencher.
- `{% if session.logado %}` mostra os links *+ Novo* e *Sair* só para quem fez login; os outros veem o link *Entrar*.
- `get_flashed_messages()` busca os avisos criados com `flash()` no `app.py` (como "Computador cadastrado"). Cada aviso aparece uma única vez.

4.2 A página inicial: lista e busca

Volte ao `app.py` e acrescente o início da parte das páginas:

app.py (parte 3 de 9)

```
# ================================================================
# PARTE 1 – PÁGINAS HTML (templates)
# ================================================================
@app.route("/")
def pagina_inicial():
    busca = request.args.get("busca", "")
    lista = filtrar(busca)
    return render_template("lista.html", computadores=lista, busca=busca)
```

A busca vem da URL. Quando o usuário digita *lab* e clica em *Buscar*, o navegador acessa `/?busca=lab` e `request.args.get("busca", "")` devolve `"lab"`. Sem busca, o valor padrão é `""` e `filtrar()` devolve todos os computadores.

templates/lista.html

```
{% block titulo %}Computadores{% endblock %}

{% block conteudo %}
  <h1>Computadores monitorados</h1>

  <form method="get" action="{{ url_for('pagina_inicial') }}">
    <input type="text" name="busca" value="{{ busca }}" placeholder="Buscar por nome, IP ou MAC">
    <button type="submit">Buscar</button>
  </form>

  {% if computadores %}
    <table>
      <tr>
        <th>ID</th><th>Nome</th><th>Endereço IP</th><th>Endereço MAC</th>
        {% if session.logado %}<th>Ações</th>{% endif %}
      </tr>
      {% for c in computadores %}
        <tr>
          <td>{{ c.id }}</td>
          <td>{{ c.nome }}</td>
          <td>{{ c.ip }}</td>
          <td>{{ c.mac }}</td>
          {% if session.logado %}
            <td>
              <a href="{{ url_for('editar_computador', id=c.id) }}">Editar</a>
              <form method="post" action="{{ url_for('apagar_computador', id=c.id) }}" style="display:inline"
                    onsubmit="return confirm('Apagar este computador?')">
                <button type="submit">Apagar</button>
              </form>
            </td>
          {% endif %}
        </tr>
      {% endfor %}
    </table>
    <p>{{ computadores|length }} computador(es) encontrado(s).</p>
  {% else %}
    <p>Nenhum computador encontrado.</p>
  {% endif %}
{% endblock %}
```

- `{% extends "base.html" %}` diz que esta página usa o molde; ela só escreve o que vai dentro dos blocos.
- O formulário de busca usa `method="get"`: os dados vão na URL, como `?busca=lab`. Usamos GET porque buscar **não altera** nada no servidor.
- `value="{{ busca }}"` mantém o texto pesquisado na caixa depois da busca.
- `{% for c in computadores %}` cria uma linha `<tr>` para cada computador. Dentro do laço, `{{ c.nome }}` acessa a chave `"nome"` do dicionário.
- O botão *Apagar* é um pequeno formulário com `method="post"`, porque apagar **altera** os dados. O `confirm()` pede uma confirmação antes de enviar.

4.3 Login e logout

app.py (parte 4 de 9)

```
@app.route("/login", methods=["GET", "POST"])
def login():
    if request.method == "POST":
        if request.form["usuario"] == USUARIO and request.form["senha"] == SENHA:
            session["logado"] = True
            flash("Login realizado com sucesso.")
            return redirect(url_for("pagina_inicial"))
        flash("Usuário ou senha incorretos.")
    return render_template("login.html")


@app.route("/logout")
def logout():
    session.clear()
    flash("Você saiu do sistema.")
    return redirect(url_for("pagina_inicial"))
```

Requisição	O que acontece
GET `/login`	O usuário clicou em *Entrar*: a função não entra no `if` e apenas mostra o formulário.
POST `/login`	O usuário enviou o formulário: `request.form` traz o que foi digitado. Se usuário e senha estiverem corretos, marcamos `session["logado"] = True` e redirecionamos para a página inicial.

- `session` é um dicionário que o Flask guarda **no navegador do usuário**, em um cookie assinado com a `secret_key`. Por isso o login continua valendo de uma página para outra.
- `redirect(url_for("pagina_inicial"))` manda o navegador ir para outra página.
- `session.clear()` apaga a sessão: o usuário deixa de estar logado.

templates/login.html

```
{% extends "base.html" %}

{% block titulo %}Entrar{% endblock %}

{% block conteudo %}
  <h1>Entrar</h1>
  <p>É preciso entrar para cadastrar, editar ou apagar computadores.</p>

  <form method="post">
    <label>Usuário
      <input type="text" name="usuario" required>
    </label>
    <label>Senha
      <input type="password" name="senha" required>
    </label>
    <button type="submit">Entrar</button>
  </form>
{% endblock %}
```

Os atributos `name="usuario"` e `name="senha"` dos campos são as chaves que aparecem em `request.form`. O campo `type="password"` esconde o que é digitado.

4.4 Cadastrar e editar (Create e Update)

app.py (parte 5 de 9)

```
    global proximo_id
    if not usuario_logado():
        return redirect(url_for("login"))

    if request.method == "POST":
        erro = validar(request.form)
        if erro:
            flash(erro)
            return render_template("formulario.html", titulo="Novo computador", dados=request.form)
        novo = limpar(request.form)
        novo["id"] = proximo_id
        proximo_id += 1
        computadores.append(novo)
        flash(f"Computador {novo['nome']} cadastrado.")
        return redirect(url_for("pagina_inicial"))

    return render_template("formulario.html", titulo="Novo computador", dados={})


@app.route("/computadores/<int:id>/editar", methods=["GET", "POST"])
def editar_computador(id):
    if not usuario_logado():
        return redirect(url_for("login"))
    computador = buscar_por_id(id)
    if computador is None:
        flash("Computador não encontrado.")
        return redirect(url_for("pagina_inicial"))

    if request.method == "POST":
        erro = validar(request.form)
        if erro:
            flash(erro)
            return render_template("formulario.html", titulo="Editar computador", dados=request.form)
        computador.update(limpar(request.form))
        flash(f"Computador {computador['nome']} atualizado.")
        return redirect(url_for("pagina_inicial"))

    return render_template("formulario.html", titulo="Editar computador", dados=computador)
```

As duas funções seguem o mesmo roteiro:

1. Se o usuário **não** fez login, é mandado para a página de login.
2. Se a requisição é **GET**, mostra o formulário (vazio no cadastro, preenchido na edição).
3. Se a requisição é **POST**, valida o que foi digitado:

- com erro: mostra o aviso e devolve o formulário **com o que o usuário já tinha digitado** (`dados=request.form`), para ele não ter que digitar tudo de novo;
- sem erro: salva, cria um aviso de sucesso e volta para a lista.

Na edição, `computador.update(limpar(request.form))` troca o nome, o IP e o MAC do dicionário que já está na lista. O `id` não muda.

templates/formulario.html

```
{% block titulo %}{{ titulo }}{% endblock %}

{% block conteudo %}
  <h1>{{ titulo }}</h1>

  <form method="post">
    <label>Nome
      <input type="text" name="nome" value="{{ dados.nome }}" required>
    </label>
    <label>Endereço IP
      <input type="text" name="ip" value="{{ dados.ip }}" placeholder="192.168.0.10" required>
    </label>
    <label>Endereço MAC
      <input type="text" name="mac" value="{{ dados.mac }}" placeholder="AA:BB:CC:DD:EE:FF" required>
    </label>
    <button type="submit">Salvar</button>
    <a href="{{ url_for('pagina_inicial') }}">Cancelar</a>
  </form>
{% endblock %}
```

O mesmo template serve para as duas operações: só mudam o `titulo` e os `dados` enviados pela rota. Como o `<form>` não tem `action`, ele é enviado para o mesmo endereço em que está (`/computadores/novo` ou `/computadores/1/editar`).

4.5 Apagar (Delete)

app.py (parte 6 de 9)

```
@app.route("/computadores/<int:id>/apagar", methods=["POST"])
def apagar_computador(id):
    if not usuario_logado():
        return redirect(url_for("login"))
    computador = buscar_por_id(id)
    if computador:
        computadores.remove(computador)
        flash(f"Computador {computador['nome']} apagado.")
    return redirect(url_for("pagina_inicial"))
```

> **Por que apagar só com POST?**
>
> Se apagar fosse um simples link (GET), bastaria alguém abrir o endereço `/computadores/1/apagar`, ou um programa que segue links automaticamente, para apagar dados. Como regra, **operações que alteram dados nunca devem usar GET**.

5. A API REST (JSON)

Agora vamos criar as rotas para programas. Elas usam as **mesmas funções auxiliares** e a **mesma lista** das páginas: um computador cadastrado pelo navegador aparece na API, e vice-versa.

5.1 Consultas: abertas a todos

app.py (parte 7 de 9)

```
# ================================================================
# PARTE 2 – API REST (JSON)
# ================================================================
@app.route("/api/computadores", methods=["GET"])
def api_listar():
    busca = request.args.get("busca", "")
    return jsonify(filtrar(busca))


@app.route("/api/computadores/<int:id>", methods=["GET"])
def api_consultar(id):
    computador = buscar_por_id(id)
    if computador is None:
        return jsonify({"erro": "Computador não encontrado."}), 404
    return jsonify(computador)
```

app.py (parte 8 de 9)

```
@app.route("/api/computadores", methods=["POST"])
def api_cadastrar():
    global proximo_id
    if not token_valido():
        return jsonify({"erro": "Token ausente ou inválido."}), 401

    dados = request.get_json(silent=True)
    erro = validar(dados)
    if erro:
        return jsonify({"erro": erro}), 400

    novo = limpar(dados)
    novo["id"] = proximo_id
    proximo_id += 1
    computadores.append(novo)
    return jsonify(novo), 201


@app.route("/api/computadores/<int:id>", methods=["PUT"])
def api_atualizar(id):
    if not token_valido():
        return jsonify({"erro": "Token ausente ou inválido."}), 401
    computador = buscar_por_id(id)
    if computador is None:
        return jsonify({"erro": "Computador não encontrado."}), 404

    dados = request.get_json(silent=True)
    erro = validar(dados)
    if erro:
        return jsonify({"erro": erro}), 400

    computador.update(limpar(dados))
    return jsonify(computador)


@app.route("/api/computadores/<int:id>", methods=["DELETE"])
def api_apagar(id):
    if not token_valido():
        return jsonify({"erro": "Token ausente ou inválido."}), 401
    computador = buscar_por_id(id)
    if computador is None:
        return jsonify({"erro": "Computador não encontrado."}), 404

    computadores.remove(computador)
    return "", 204
```

Repare que cada função de alteração começa conferindo o token. Se ele estiver ausente ou errado, a função para ali mesmo e responde **401 Unauthorized**, sem tocar nos dados.

Situação	Código	Corpo da resposta
Consulta bem-sucedida	200	O computador ou a lista
Cadastro bem-sucedido	201	O computador criado, com o `id`
Exclusão bem-sucedida	204	Vazio
Dados inválidos	400	`{"erro": "..."}`
Token ausente ou errado	401	`{"erro": "Token ausente ou inválido."}`
Id inexistente	404	`{"erro": "Computador não encontrado."}`

5.3 Final do arquivo

app.py (parte 9 de 9)

```
if __name__ == "__main__":
    app.run(debug=True)
```

> **Duas formas de autenticação, por quê?**

> `Authorization`, como fizemos com a REST Countries.

6. Testando

6.1 No navegador

Execute `python app.py` e acesse `http://127.0.0.1:5000`.

1. Veja a lista com os 3 computadores iniciais e faça uma busca por *lab* ou por *00:03*.
2. Clique em *Entrar* e use o usuário `admin` e a senha `ifrn`.
3. Cadastre um computador com o MAC `aa-bb-cc-11-22-33` e veja que ele aparece como `AA:BB:CC:11:22:33`.
4. Tente cadastrar com o IP `192.168.0.300` e veja a mensagem de erro.
5. Edite e apague um computador. Depois clique em *Sair* e veja que os botões de edição desaparecem.
6. Acesse `http://127.0.0.1:5000/api/computadores` e `http://127.0.0.1:5000/api/computadores?busca=lab`: é a mesma informação, agora em JSON.

6.2 A API com curl

O `curl` já vem instalado no Linux, no macOS e no Windows 10/11. Em outro terminal:

```
curl http://127.0.0.1:5000/api/computadores/1

curl -X DELETE http://127.0.0.1:5000/api/computadores/1

curl -X DELETE -H "Authorization: Bearer token-da-turma" -i http://127.0.0.1:5000/api/computadores/1
```

```
{
  "id": 1,
  "ip": "192.168.0.10",
  "mac": "AA:BB:CC:00:00:01",
  "nome": "Servidor-Web"
}

{
  "erro": "Token ausente ou inválido."
}

HTTP/1.1 204 NO CONTENT
...
```

O primeiro `DELETE` foi recusado por falta do token; o segundo apagou o computador. *O Flask coloca as chaves do JSON em ordem alfabética; isso não muda nada para quem lê os dados pelo nome.*

> **Atenção no Windows**
>
> No PowerShell antigo, `curl` pode ser um apelido para outro comando. Se a saída parecer estranha, use `curl.exe` no lugar de `curl`.

7. O cliente em modo texto (cliente.py)

Por fim, um programa com menu que consome a API, como fizemos na Unidade 2 com a biblioteca `requests`. Deixe o `app.py` rodando em um terminal e execute o cliente em outro.

cliente.py

```
import requests

URL = "http://127.0.0.1:5000/api/computadores"
token = ""   # será pedido na primeira operação que precisar dele


def cabecalho_autorizacao():
    global token
    if token == "":
        token = input("Token da API: ").strip()
    return {"Authorization": f"Bearer {token}"}


def mostrar_tabela(lista):
    if len(lista) == 0:
        print("Nenhum computador encontrado.")
        return
    print(f"{'ID':<4} {'NOME':<20} {'IP':<16} {'MAC':<17}")
    print("-" * 60)
    for c in lista:
        print(f"{c['id']:<4} {c['nome']:<20} {c['ip']:<16} {c['mac']:<17}")
    print(f"Total: {len(lista)}")


def mostrar_erro(resposta):
    global token
    if resposta.status_code == 401:
        token = ""   # esquece o token errado para pedir de novo na próxima vez
    try:
        mensagem = resposta.json()["erro"]
    except (ValueError, KeyError):
        mensagem = resposta.reason
    print(f"Erro {resposta.status_code}: {mensagem}")


def listar():
    resposta = requests.get(URL, timeout=5)
    mostrar_tabela(resposta.json())


def buscar():
    texto = input("Buscar (nome, IP ou MAC): ")
    resposta = requests.get(URL, params={"busca": texto}, timeout=5)
    mostrar_tabela(resposta.json())


def consultar():
    id = input("ID do computador: ")
    resposta = requests.get(f"{URL}/{id}", timeout=5)
    if resposta.status_code == 200:
        c = resposta.json()
        print(f"Nome: {c['nome']}\nIP:   {c['ip']}\nMAC:  {c['mac']}")
    else:
        mostrar_erro(resposta)


def ler_dados():
    return {
        "nome": input("Nome: "),
        "ip": input("Endereço IP: "),
        "mac": input("Endereço MAC: "),
    }


def cadastrar():
    dados = ler_dados()
    resposta = requests.post(URL, json=dados, headers=cabecalho_autorizacao(), timeout=5)
    if resposta.status_code == 201:
        print("Cadastrado com o ID", resposta.json()["id"])
    else:
        mostrar_erro(resposta)


def editar():
```

```
    if resposta.status_code == 200:
        print("Atualizado com sucesso.")
    else:
        mostrar_erro(resposta)


def apagar():
    id = input("ID do computador: ")
    resposta = requests.delete(f"{URL}/{id}", headers=cabecalho_autorizacao(), timeout=5)
    if resposta.status_code == 204:
        print("Apagado com sucesso.")
    else:
        mostrar_erro(resposta)


MENU = """
===== MONITORAMENTO =====
1 - Listar todos
2 - Buscar
3 - Consultar por ID
4 - Cadastrar   (token)
5 - Editar      (token)
6 - Apagar      (token)
0 - Sair"""

while True:
    print(MENU)
    opcao = input("Opção: ").strip()
    try:
        if opcao == "1":
            listar()
        elif opcao == "2":
            buscar()
        elif opcao == "3":
            consultar()
        elif opcao == "4":
            cadastrar()
        elif opcao == "5":
            editar()
        elif opcao == "6":
            apagar()
        elif opcao == "0":
            print("Até mais!")
            break
        else:
            print("Opção inválida.")
    except requests.exceptions.ConnectionError:
        print("Não foi possível conectar. O app.py está em execução?")
```

Entendendo o cliente

- Cada opção do menu é uma função. O laço `while True` mostra o menu, lê a opção e chama a função correspondente com `if`/`elif`.
- As consultas (`listar`, `buscar`, `consultar`) não enviam token.
- As alterações chamam `cabecalho_autorizacao()`, que pede o token só na primeira vez e depois o reaproveita. Se a API responder 401, `mostrar_erro()` apaga o token guardado, para que ele seja pedido de novo.
- `mostrar_tabela()` usa a formatação `{valor:<20}` para alinhar as colunas (texto à esquerda, ocupando 20 posições).
- O `try/except` no laço principal evita que o programa feche com erro se o servidor estiver desligado.

Exemplo de uso

```
3 - Consultar por ID
4 - Cadastrar   (token)
5 - Editar      (token)
6 - Apagar      (token)
0 - Sair
Opção: 2
Buscar (nome, IP ou MAC): lab
ID   NOME                IP              MAC
------------------------------------------------------------
2    Lab-PC-01           192.168.0.101   AA:BB:CC:00:00:02
Total: 1

Opção: 4
Nome: Switch-Core
Endereço IP: 192.168.0.2
Endereço MAC: aa-bb-cc-00-00-09
Token da API: errado
Erro 401: Token ausente ou inválido.

Opção: 4
Nome: Switch-Core
Endereço IP: 192.168.0.2
Endereço MAC: aa-bb-cc-00-00-09
Token da API: token-da-turma
Cadastrado com o ID 4
```

Atualize a página no navegador: o *Switch-Core* cadastrado pelo cliente aparece lá também.

8. Limitações desta versão

Para manter o código simples, este sistema tem limitações que um sistema real **não pode** ter:

Nesta aula	Em um sistema real
Senha, token e `secret_key` escritos no código	Lidos de variáveis de ambiente ou de um arquivo de configuração fora do repositório
Um único usuário com senha em texto	Usuários em um banco de dados, com senhas guardadas como *hash*
Dados em uma lista (somem ao reiniciar)	Banco de dados (SQLite, PostgreSQL...)
HTTP	HTTPS, para que senha e token não trafeguem às claras na rede
Servidor de desenvolvimento com `debug=True`	Servidor WSGI de produção (Gunicorn, Waitress)

9. Exercícios

1. **Página de detalhes:** crie a rota `/computadores/<int:id>` e o template `detalhe.html`, que mostra os dados de um único computador. Na lista, transforme o nome em um link para essa página.
2. **Sem repetidos:** impeça o cadastro de dois computadores com o mesmo MAC. Na API, responda com o código **409 Conflict**.
3. **Novo campo:** acrescente o campo `setor` (ex.: *Laboratório 1*). Altere o formulário, a tabela, a validação, a busca e o cliente.
4. **Contador:** mostre na página inicial quantos computadores estão na faixa `192.168.0.`. Dica: faça a conta no `app.py` e envie o resultado ao template.
5. **Desafio:** no cliente, crie a opção *7 - Exportar*, que salva a lista de computadores em um arquivo `computadores.csv`.

> **Entrega**
>
> Compacte a pasta `monitor` (sem a pasta `.venv`) e envie o arquivo `.zip`.

Resumo

Conceito	Onde aparece
Template e herança	`render_template()`, `{% extends %}`, `{% block %}`
Valores, condições e repetições no HTML	`{{ }}`, `{% if %}`, `{% for %}`
Dados de formulário	`request.form["nome"]`
Dados da URL (busca)	`request.args.get("busca")`

Login	`session`, `redirect()`, `url_for()`
Avisos ao usuário	`flash()` e `get_flashed_messages()`
Autenticação da API	Cabeçalho `Authorization: Bearer ...` e código 401
CRUD	Create (POST), Read (GET), Update (PUT/POST), Delete (DELETE/POST)

Última atualização: quinta-feira, 29 out. 2026, 18:36

◄ Aula 20251027 - consumindo APIs públicas com Python (PokeAPI e REST Countries)

Ir para a atividade